Modeling Human Activity States Using Hidden Markov Models

Github Repository and Google Colab Link

Folder with videos

1. Background and motivation

My use case is checking on older people who live alone, so their phone can work out whether they are still moving around normally. The hard part is telling a phone lying on a table apart from a phone held by someone standing. Mix those up and the system either cries wolf every time the phone is put down, or stays quiet when the person has stopped moving. The phone is the only sensor these people reliably carry, and the activity is never measured directly. An HMM suits this because activity does not jump around at random: someone walking now is very likely still walking two seconds later, and using that lets the model fix single windows it would otherwise get wrong.

2. Data collection and pre-processing

All data came from one iPhone 11 Pro (iOS 18.7.7) running Sensor Logger 1.68, logging accelerometer, gyroscope and gravity with every other sensor off.

Sampling rate. All three sensors were set to 50 Hz. Measured across the 26 recordings the rate was 50.194 Hz (50.190 to 50.204), with worst-case jitter of 0.0439 ms, 0.219% of the 19.93 ms sample interval. I picked 50 Hz from Nyquist: human movement sits below about 20 Hz, walking near 1.8 Hz and jumping near 1.9 Hz, so 50 Hz covers everything to 25 Hz.

Different sampling rates. With one phone there were no other rates to line up, but the notebook does not trust the setting: it reads each session's timestamps, works out the real rate and jitter, and writes them to a manifest. A session at a different rate would be flagged and resampled onto a common grid before windowing, which is the step that would handle several phones. All three sensors share identical timestamps in every recording, so merging is just joining columns.

Recording and cleaning. For Standing, Walking and Jumping I held the phone in my right hand at waist level, upright in portrait; for Still I put it flat on a table and walked away. I walked at about 105 steps per minute. Training was recorded on 25 October 2026, test on 26 October 2026. I threw out 6 of 26 recordings: four made before I settled on the grip, with the phone flat in my palm, which puts gravity on the z axis, the same axis as a phone on a table, and would have made Standing look like Still; two duplicate exports; and one walking session recorded at 73 steps per minute. I then cut 1.5 s off both ends of every session, since those moments are me reaching for the record button.

Videos. Standing · Walking · Jumping · Still

Activity	Sessions	Training files	Duration (s)	Test files
Still	4	24	164	3
Standing	4	24	166	3
Walking	3	18	124	3

Activity	Sessions	Training files	Duration (s)	Test files
Jumping	5	16	110	3
Total	**16**	**82**	**564**	**12**

Each session was cut into 7-second labelled CSV files, giving 82 training and 12 test files, every activity past the 90-second minimum.

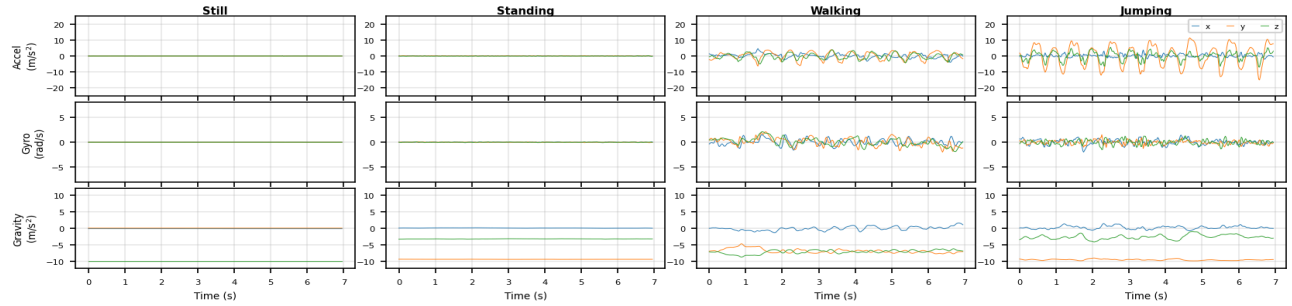


Figure 1. Raw signals for one window of each activity on shared axes. Jumping reaches ±15 m/s², walking ±5, and the two low-motion activities sit flat near zero. Zoomed 100×, Standing shows mean ‖a‖ of 0.1012 m/s² against 0.0119 for Still, and the gravity row shows the two loading different axes.

3. Feature extraction

Window size follows from the sampling rate. Features are computed over 4-second windows, 200 samples at 50 Hz, at 50% overlap. Frequency resolution is one over the window length, so a 2-second window resolves only 0.5 Hz, useless when walking sits at 1.74 Hz and jumping at 1.95 Hz. A 4-second window gives 0.25 Hz. I checked this on real data: 4-second windows returned 1.76 Hz every time on one walking session, while 2-second windows on the same signal bounced between 0.50 and 3.51 Hz. The 82 training files gave 161 windows and the 12 test files 24.

Vertical acceleration. Several features use acceleration projected onto the gravity direction rather than the magnitude ‖a‖, which throws away the sign and makes the signal look twice as fast: on my walking data ‖a‖ reported 3.57 Hz when the real cadence was 1.75 Hz.

Domain	Features and why each helps
Time (6)	**var_vert** 0.001 (Still) to 74.9 (Jumping), the strongest single feature. **rms_acc** movement energy, 0.020 to 8.674. **sma** signal magnitude area, barely affected by phone tilt. **gyro_mean** 0.003 Still against 0.053 Standing, a 17 times gap independent of acceleration. **range_vert** largest up-down swing, catching the hard landings in Jumping. **corr_xy** walking couples the axes, noise at rest does not.
Orientation (3)	**grav_x, grav_y, grav_z** upright puts gravity on y at minus 9.39, flat on a table on z at minus 9.806. This is what splits the two still-looking activities.

Domain	Features and why each helps
Frequency, from FFT (3)	**dom_freq** cadence from the FFT peak between 0.3 and 12 Hz: 1.74 Hz walking, 1.95 Hz jumping. **spec_energy** 0.021 to 5678, four orders of magnitude between rest and movement. **spec_entropy** rhythmic motion 0.40, sensor noise 0.89.

A Hann window is applied before the FFT to limit leakage, and the DC component removed so bin 0 does not win the peak search.

Normalisation. Six features are energy-like and span five orders of magnitude, so I first squash them with log(1+|x|); without this, variance alone would swamp the Gaussian emission likelihoods and the orientation and frequency features would count for nothing. All 12 are then Z-scored to zero mean and unit variance. I chose Z-score over min-max because each feature is roughly unimodal within a class, and min-max would let one freak window squeeze everything else into a narrow band. The mean and standard deviation come from training data only, so nothing about the test set leaks in.

4. HMM setup and implementation

The hidden states Z are the four activities. The observations X are the 12-number normalised feature vector per window. A is the 4×4 matrix of the chance of each activity next window given the current one, B is one Gaussian per state with diagonal covariance (4 × 12 means and variances), and π is the chance a sequence starts in each activity. I used Gaussian emissions because the features are continuous, and kept the covariance diagonal because 161 windows cannot support the 78 free covariance terms per state a full matrix would need.

Which algorithms fit, and how they relate to the metrics. The **forward algorithm** gives $P(X \mid \lambda)$, how likely a sequence is. It produces no labels, so it feeds no metric directly, but it is the number Baum-Welch watches to decide when to stop. **Baum-Welch** solves the learning problem, re-estimating A, B and π by expectation-maximisation, so it sets the quality of the parameters every metric depends on. **Viterbi** solves the decoding problem and produces the predictions, so sensitivity, specificity and accuracy all measure Viterbi. Viterbi beats picking the best state per window because windows are not independent. I report specificity beside accuracy because the classes are uneven, 46 down to 31 windows under cross-validation, and accuracy alone can hide errors concentrated in the smallest class.

Numerical work. Everything is from scratch in NumPy with no HMM library. Forward, backward and Viterbi run on log probabilities, because in ordinary space the forward variable underflows to zero within a few dozen steps, after which Baum-Welch divides by zero and training quietly produces rubbish while still appearing to run. Probabilities are added with a max-shifted log-sum-exp. Each iteration runs forward-backward over every sequence, collects the posterior occupancies γ and transition posteriors ξ, then re-estimates π, A, μ and σ^2 in closed form. **Training stops when the total log-likelihood changes by less than ε = 1e-4**, not after a fixed number of loops; a 200-iteration ceiling was never reached. A variance floor of 1e-2 stops a near-constant state collapsing to zero variance, and a probability floor of 1e-6 keeps log(0) out of the recursions. I ran training twice, seeded from the labelled windows and from random parameters, which shows the EM code is really optimising rather than sitting on a good starting guess.

5. Results and interpretation

Convergence. From the labelled seeds, Baum-Welch converged in 3 iterations at a log-likelihood of 748.87. From random parameters it climbed from minus 2489 over 37 iterations to 287.19. Both met the ε test. That the random start converges tells me the EM code works; that it lands lower tells me it found a worse local optimum, which is what EM does, since it only promises to go uphill and not to find the highest hill. I kept the model from the labelled start.

Unseen data. The test set is four sessions recorded on 26 October 2026, the day after all training data, in a different room. No window from a test session went into training, and the normalisation statistics were fixed beforehand.

State (Activity)	Number of Samples	Sensitivity	Specificity	Overall Accuracy
Still	6	1.000	1.000	1.000
Standing	6	1.000	1.000	1.000
Walking	6	1.000	1.000	1.000
Jumping	6	1.000	1.000	1.000

Cross-validation. Twenty-four windows felt thin as a basis for a perfect score, so I also ran leave-one-session-out cross-validation across the 16 training sessions, retraining from scratch on 15 and testing on the one left out. That gave 161 windows (46 Still, 48 Standing, 36 Walking, 31 Jumping) and also returned 1.000 on all three metrics for every activity.

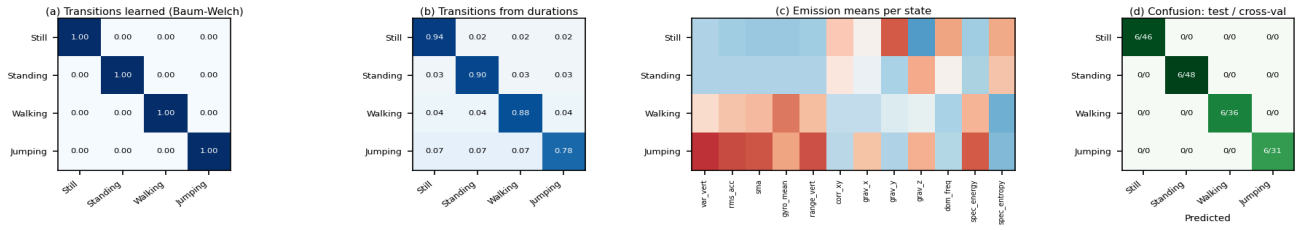


Figure 2. (a) The transition matrix Baum-Welch learned, a degenerate identity. (b) The duration-based alternative. (c) Emission means per state across the 12 features. (d) Confusion matrices, shown as test / cross-validation counts; both are purely diagonal.

Decoded sequences. Viterbi got every window right on the test sequences. Because the test files are grouped by activity I also built a composite session in the order Still → Standing → Walking → Jumping → Walking → Standing → Still, giving 14 windows with six real activity changes. Viterbi found all seven segments and put every change in the right window.

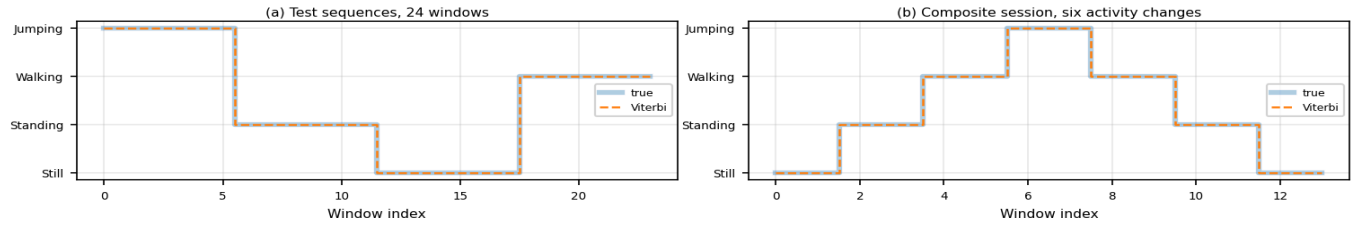


Figure 3. Decoded sequences. The dashed Viterbi path sits exactly on the true path in both panels.

6. Discussion and conclusion

Why everything came out perfect. I do not think this is leakage: the normalisation statistics came from training data only, and each held-out session was kept out of its own training run. The honest explanation is that my problem is easy at this resolution. Variance of vertical acceleration alone goes 0.001, 0.004, 6.17, 74.86 across the four activities, and the two that share a low variance are split apart by gravity sitting on different axes (Figure 2c). A decision tree on two features would probably score near 1.000 too. The result says my pipeline is correct, not that the model is doing anything clever, and the numbers would not survive one person, one phone and one fixed grip being relaxed.

Easiest and hardest to tell apart. Jumping was easiest by a long way, an order of magnitude past anything else at 74.86 vertical variance. I expected Still and Standing to be hardest, since both barely move and are only 0.09 m/s² apart in mean acceleration. They separated cleanly because three independent things agree: Standing has about 9 times the acceleration, 15 times the rotation, and gravity on a different axis. Gravity is the dependable one, because it reads which way the phone points rather than how much it moves. The closest pair by cadence is Walking and Jumping, 1.74 against 1.95 Hz, within one FFT bin, and those are told apart by amplitude instead.

What the transition probabilities say. The matrix Baum-Welch learned (Figure 2a) is an identity, every self-transition 1.000. That is a fact about my data, not about behaviour: each training file holds one activity start to finish, so the model never saw a transition, and a model with this A can never change state. To show realistic transitions I built a matrix from expected durations (Figure 2b). With a 2-second window step and mean holding times of 30 s Still, 20 s Standing, 15 s Walking and 8 s Jumping, the self-transitions come out at 0.936, 0.905, 0.875 and 0.779. All are high because activity lasts far longer than one window, and Jumping is lowest because it is given up on quickest. Decoding the composite session with this matrix still got every transition right. Learning A properly would need recordings where I change activity mid-session.

Sensor noise and sampling rate. Timing was better than expected: 0.0439 ms worst-case jitter with identical timestamps across all three sensors, which removed the resampling step I had planned. Noise mattered most in Still, where the accelerometer sits at 0.0119 m/s², essentially the sensor noise floor. That helps rather than hurts: its spectral entropy of 0.889 reflects a flat, shapeless spectrum, exactly what noise looks like and what sets it apart from rhythmic movement. The sampling rate shaped everything downstream, because 50 Hz fixed 200 samples per window, which fixed the 0.25 Hz resolution, which was the limit on separating walking from jumping.

Conclusion and improvements. Three decisions mattered, and the data forced all three: the window had to be 4 seconds, because 2 seconds cannot resolve the 0.21 Hz gap between walking and jumping; dominant frequency had to come from gravity-projected vertical acceleration, because the magnitude reports double the real cadence; and gravity had to be an explicit feature, because it separates a phone held upright from one on a table without depending on how still the person stands. More people and phone placements would be the biggest improvement, and overlapping activities such as climbing stairs would make the problem hard enough that the HMM's use of time earns its place.